Softwareparadigms SS 2025, Assignment 3

Submission: The solution has to be submitted via the SAI student git. Using the provided latex template is mandatory (No handwritten solutions!). Please check in the LaTeX source **as well as** the compiled PDF. Both files have to follow the naming convention '`sol<assignment number>_<group number>.<ext>`' (e.g. '`sol3_23.tex`').
Deadline Assignment 3: 04.06.2025, 17:00 Uhr
Remarks for AL/ALP:

- If the data type corresponds to a class of numbers (such as the natural numbers $\mathbb{N}$), the set of function symbols $\{\underline{+}, \underline{-}, \underline{*}\}$ and the set of predicate symbols $\{\underline{lt?}, \underline{lteq?}, \underline{eq?}, \underline{gt?}, \underline{gteq?}\}$ are assigned to the respective mathematical operators $+, -, *, <, \leq, =, >, \geq$. The numbers may be used as constants according to the decimal system.
- The infix notation and interpretation commonly used in arithmetic may be used in the target set of the interpretation function via the data type, provided that it is correct and unambiguous.
- Specify intermediate steps of the interpretation function. Individual code sections can be abbreviated with $LX - LY$, where X and Y stand for the corresponding line numbers.
- The following notation can be used for nested interpretation functions of ALP: $I_{ALP}(I_{ALP}(\omega, \sigma, p_1), p_2)$. This simplification of the notation $I_{ALP}(I_{ALP}(\omega, \sigma, p_1)(\omega), I_{ALP}(\omega, \sigma, p_1)(\sigma), p_2)$ is allowed in this course, since the interpretation function of ALP returns a tuple (ω, σ).

Remarks for LP:

- LP $\neq$ Prolog! Only facts and rules of the form: $A := A_1, ..., A_k$ should be used to solve the examples, where A and $A_1, ..., A_k$ are atomic formulas. Therefore, LP programs defined by you must not contain any cuts, prolog operators, etc.
  Attention: `\=` is also a prolog operator that does not exist in LP!
- Follow the format of the LaTeXtemplate for proofs using Resultion.
- Always specify the following fields for a resolution step:
  - Q: the query to be processed
  - R: the rule used including identifier, e.g. R(A2) for the second rule of `add(X,Y,Z)`
  - Θ: the substitution
  - QΘ: the substituted query, i.e. the query after applying the substitution Θ
  - RΘ: the substituted rule, i.e. the rule after applying the substitution Θ

  If no variables are substituted in the query or rule, you can omit the corresponding field (QΘ or RΘ).
- The scope of a variable only affects the clause in which it occurs directly. Therefore, do not forget to use a separate namespace for each rule! To do this, introduce new variables when you convert a rule into a clause in order to use it for a resolution by marking the variables from the rule with subscripts.

Task 1: AL (2 P.)

1. Write a small program p in the language AL over the datatype of natural numbers, which - given two numbers a and b, calculates the result of the integer division (a *div* b) and the remainder (a % b), where a is the dividend and b the divisor. Note that neither integer division nor the remainder are available as built-ins. You can assume that the numbers a and b are provided in the initial environment ω_0 using the variable names $\underline{\mathtt{a}}$ and $\underline{\mathtt{b}}$. Formally, the following should hold for your program:
   - $I_{AL}(\omega^0, p)(\underline{\mathtt{t}}) = \omega^0(\underline{\mathtt{a}})\ div\ \omega^0(\underline{\mathtt{b}})$ and
   - $I_{AL}(\omega^0, p)(\underline{\mathtt{r}}) = \omega^0(\underline{\mathtt{a}})\ \%\ \omega^0(\underline{\mathtt{b}})$
2. Evaluate your program using $\omega^0(\underline{\mathtt{a}}) = 5$ and $\omega^0(\underline{\mathtt{b}}) = 3$. Write down the complete environment after executing your program.

1.1 Program p:

```
begin
    begin
        t := 1;
        r := a;
    end;
    while r >= b do
        r = r - b;
        t = t + 1;
    od;
end
```

1.2 Evaluation:

Step	Line executed	Environment ω	Evaluation
0	$t := 0$	t = 0	
1	$r := a$	r = 5	
2	WHILE ($5 \geq 3$)	unchanged	true
	$r := r - b$ (5–3)	r = 2	
	$t := t + 1$ (0+1)	t = 1	
3	WHILE ($2 \geq 3$)	loop terminates	false

The final environment is:

$$\boxed{\omega^1 = \omega^4 = \{\, a = 5,\ b = 3,\ t = 1,\ r = 2 \,\}}$$

$$I_{AL}(\omega^0, p)(t) = 1 = 5 \operatorname{div} 3,$$
$$I_{AL}(\omega^0, p)(r) = 2 = 5 \bmod 3.$$

Both required equalities hold.

Task 2: $\mathbf{AL^P}$ (1 P.)

Interpret the following program $p2$ using the interpretation function for $\mathbf{AL^P}$ presented in the lecture. For the initial environment, the following holds: $\omega^0(\underline{\mathtt{x}}) \neq \omega^0(\underline{\mathtt{y}})$. Write down the complete final environment ω and the complete final store σ.

```
1  begin
2    begin
3      x := 1;
4      ref y := ref x
5    end;
6    begin
7      y := 10;
8      if gt?(x,1) then
9          x := *(x,y);
10     else
11         x := 0;
12   end
13 end
```

Initial configuration.

$$\omega_0 : x \mapsto L_x,\ y \mapsto L_y, \qquad L_x \neq L_y, \qquad \sigma_0(L_x) = v_x^0,\ \sigma_0(L_y) = v_y^0.$$

Block 1: lines 2-5

$$\mathsf{ALP}\big(\omega_0, \sigma_0, \mathsf{begin}\ L2 - L5\ \mathsf{end}\big) \ = \ (\omega^{(4)}, \sigma^{(3)}).$$

$$\mathsf{ALP}\big(\omega_0, \sigma_0, \mathsf{begin\ x := 1;\ \ ref\ y := ref\ x\ end}\big) \ = \ (\omega^{(4)}, \sigma^{(3)}).$$

L3. x := 1

$\sigma^{(3)} = \sigma_0[\, L_x \mapsto 1\,], \qquad \omega$ - unchanged.

L4. ref y := ref x

$\omega^{(4)} = \omega_0[\, y \mapsto L_x\,], \qquad \sigma$ - unchanged.

$$d\omega^{(4)} : x \mapsto L_x,\ y \mapsto L_x \quad | \quad \sigma^{(3)} : L_x \mapsto 1,\ L_y \mapsto v_y^0$$

Block 2: lines 6-12

$$\mathsf{ALP}\big(\omega^{(4)}, \sigma^{(3)}, \mathsf{begin}\ L6 - L12\ \mathsf{end}\big) = (\omega_{\text{fin}}, \sigma_{\text{fin}}).$$

$$\mathsf{ALP}\big(\omega^{(4)}, \sigma^{(3)}, \mathsf{begin\ y := 10;\ if\ gt?(x,1)\ then\ x := *(x,y)\ else\ x := 0\ end}\big) = (\omega_{\text{fin}}, \sigma_{\text{fin}}).$$

L7. y := 10

$\sigma^{(5)} = \sigma^{(3)}[\, L_x \mapsto 10\,].$

L8. Check the condition gt?(x,1). x = 10, which is greater than 1, hence *true*, so we proceed to the Line 9.

L9. x := *(x,y) Both arguments evaluate to 10; product $10 \times 10 = 100$.

$\sigma^{(6)} = \sigma^{(5)}[\, L_x \mapsto 100\,].$

The else-branch is skipped;

Final state.

$$\boxed{\omega_{\text{fin}} = \{\, x \mapsto L_x,\ y \mapsto L_x\,\}, \qquad \sigma_{\text{fin}} = \{\, L_x \mapsto 100,\ L_y \mapsto v_y^0\,\}}$$

Task 3: MGU (1 P.)

Calculate, if possible, the **Most General Unifier (MGU)** for the following atomic propositions. Write down every step (including the substitution). If there is no MGU, explain why.

1. **t = split(bld(s(0),nil),s(s(0)),S,G)** and **t' = split(bld(X,L),P,0,G)**

t = split(bld(s(0),nil),s(s(0)),S,G)	t' = split(bld(X,L),P,0,G)
$\Theta = \{$ X $\mid s(0)\}$	
t = split(bld(s(0),nil),s(s(0)),S,G)	t' = split(bld(s(0),L),P,0,G)
$\Theta = \{$ X $\mid s(0), L \mid nil\}$	
t = split(bld(s(0),nil),s(s(0)),S,G)	t' = split(bld(s(0),nil),P,0,G)
$\Theta = \{$ X $\mid s(0), L \mid nil, P \mid s(s(0))\}$	
t = split(bld(s(0),nil),s(s(0)),S,G)	t' = split(bld(s(0),nil),s(s(0)),0,G)
$\Theta = \{$ X $\mid s(0), L \mid nil, P \mid s(s(0)), S \mid 0\}$	
t = split(bld(s(0),nil),s(s(0)),S,G)	t' = split(bld(s(0),nil),s(s(0)),0,G)
$\Theta = \{$ X $\mid s(0), L \mid nil, P \mid s(s(0)), S \mid 0, G \mid G\}$	
unified	

2. **t = part(bld(s(s(0)),bld(s(0),nil)),s(0),X,Y)** and **t' = part(bld(A,B),P,C,bld(A,D))**

part(bld(s(s(0)),bld(s(0),nil)),s(0),X,Y)	part(bld(A,B),P,C,bld(A,D))
$\Theta = \{$ A $\mid s(s(0))\}$	
part(bld(s(0)),bld(s(0),nil)),s(0),X,Y)	part(bld(s(s(0)),B),P,C,bld(A,D))
$\Theta = \{$ A $\mid s(s(0)), B \mid bld(s(0), nil)\}$	
part(bld(s(0)),bld(s(0),nil)),s(0),X,Y)	part(bld(s(s(0)),bld(s(0),nil),P,C,bld(A,D))
$\Theta = \{$ A $\mid s(s(0)), B \mid bld(s(0), nil), P \mid s(0)\}$	
part(bld(s(0)),bld(s(0),nil)),s(0),X,Y)	part(bld(s(s(0)),bld(s(0),nil),s(0),C,bld(A,D))
$\Theta = \{$ A $\mid s(s(0)), B \mid bld(s(0), nil), P \mid s(0), X \mid C\}$	
part(bld(s(0)),bld(s(0),nil)),s(0),X,Y)	part(bld(s(s(0)),bld(s(0),nil),s(0),X,bld(A,D))
$\Theta = \{$ A $\mid s(s(0)), B \mid bld(s(0), nil), P \mid s(0), X \mid C, Y \mid bld(s(s(0)), D)\}$	
part(bld(s(0)),bld(s(0),nil)),s(0),X,bld(s(s(0)),D)	part(bld(s(s(0)),bld(s(0),nil),s(0),X,bld(s(s(0)),D))
unified	

3. **t = bld(0,bld(s(0),Y))** and **t' = bld(X,bld(Y,bld(0,nil)))**

t = bld(0,bld(s(0),Y))	t' = bld(X,bld(Y,bld(0,nil)))
$\Theta = \{$X $\mid 0\}$	
t = bld(0,bld(s(0),Y))	t' = bld(0,bld(Y,bld(0,nil)))
$\Theta = \{$X $\mid 0, Y \mid s(0)\}$	
t = bld(0,bld(s(0), s(0))	t' = bld(0,bld(s(0),bld(0,nil)))
not unified because $s(0) \neq bld(0, nil)$	

4. **t = sum(X,A,s(X),B)** and **t' = sum(s(D),s(D),A,C)**

t = sum(X,A,s(X),B)		t' = sum(s(D),s(D),A,C)
	$\Theta = \{$ X $\mid s(D)\}$	
t = sum(s(D),A,s(s(D)),B)		t' = sum(s(D),s(D),A,C)
	$\Theta = \{$ X $\mid s(D), A \mid s(D)\}$	
t = sum(s(D),s(D),s(s(D)),B)		t' = sum(s(D),s(D),A,C)
	not unified because $s(s(D) \neq s(D)$	

5. **t = result(A,B,sum(A))** and **t' = result(0,s(0),sum(X,Y))**

t = result(A,B,sum(A))		t' = result(0,s(0),sum(X,Y))
	$\Theta = \{$ A| 0$\}$	
t = result(0,B,sum(0))		t' = result(0,s(0),sum(X,Y))
	$\Theta = \{$ A| $0, B \mid s(0)\}$	
t = result(0,s(0),sum(0))		t' = result(0,s(0),sum(X,Y))
	not unified because $sum(0) \neq sum(X, Y)$	

Task 4: LP - Tic-tac-toe (1.5 P.)

For the well-known game Tic-tac-toe[1] we define the predicate **piece(X, Y, P)** which states that a piece of player **P** $\in \{nought, cross\}$ is at position **X, Y** $\in (0..2)$. The coordinates **X** and **Y** are modeled using the *successor* definition (e.g. 0, s(0)).

1. Model the game state given in Table 1 using the **piece** predicate.
2. Define the predicate **win(P)**, which is true if player **P** won the game.
3. Use resolution to show that the predicate **win(*nought*)** is true given the game state in 1.

O	X	X
O	O	
O	X	X

Table 1: Game state

Solution

4.2.1 Model the game state using the `piece` predicate

- `piece(0, 0, nought).`
- `piece(0, s(0), cross).`
- `piece(0, s(s(0)), cross).`
- `piece(s(0), 0, nought).`
- `piece(s(0), s(0), nought).`
- `piece(s(s(0)), 0, nought).`
- `piece(s(s(0)), s(0), cross).`
- `piece(s(s(0)), s(s(0)), cross).`

4.2.2 Define the predicate `win(P)`

The predicate `win(P)` is true if player `P` has three pieces in straight line, like a row, a collumn or a diagnoal

Winning Rows:

- `win(P) :- piece(0, 0, P), piece(0, s(0), P), piece(0, s(s(0)), P).`
- `win(P) :- piece(s(0), 0, P), piece(s(0), s(0), P), piece(s(0), s(s(0)), P).`
- `win(P) :- piece(s(s(0)), 0, P), piece(s(s(0)), s(0), P), piece(s(s(0)), s(s(0)), P).`

Winning Columns:

- `win(P) :- piece(0, 0, P), piece(s(0), 0, P), piece(s(s(0)), 0, P).`
- `win(P) :- piece(0, s(0), P), piece(s(0), s(0), P), piece(s(s(0)), s(0), P).`
- `win(P) :- piece(0, s(s(0)), P), piece(s(0), s(s(0)), P), piece(s(s(0)), s(s(0)), P).`

Winning Diagonals:

- `win(P) :- piece(0, 0, P), piece(s(0), s(0), P), piece(s(s(0)), s(s(0)), P).`
- `win(P) :- piece(0, s(s(0)), P), piece(s(0), s(0), P), piece(s(s(0)), 0, P).`

[1] `https://en.wikipedia.org/wiki/Tic-tac-toe`

4.2.3 Resolution

Facts (F):

- F1: `piece(0, 0, nought).`
- F2: `piece(0, s(0), cross).`
- F3: `piece(0, s(s(0)), cross).`
- F4: `piece(s(0), 0, nought).`
- F5: `piece(s(0), s(0), nought).`
- F6: `piece(s(s(0)), 0, nought).`
- F7: `piece(s(s(0)), s(0), cross).`
- F8: `piece(s(s(0)), s(s(0)), cross).`

Rules (converted to clauses, separate namespace for each rule):

- R1: $\texttt{win}(P_1) \lor \neg\texttt{piece}(0, 0, P_1) \lor \neg\texttt{piece}(0, s(0), P_1) \lor \neg\texttt{piece}(0, s(s(0)), P_1)$
- R2: $\texttt{win}(P_2) \lor \neg\texttt{piece}(s(0), 0, P_2) \lor \neg\texttt{piece}(s(0), s(0), P_2) \lor \neg\texttt{piece}(s(0), s(s(0)), P_2)$
- R3: $\texttt{win}(P_3) \lor \neg\texttt{piece}(s(s(0)), 0, P_3) \lor \neg\texttt{piece}(s(s(0)), s(0), P_3) \lor \neg\texttt{piece}(s(s(0)), s(s(0)), P_3)$
- R4: $\texttt{win}(P_4) \lor \neg\texttt{piece}(0, 0, P_4) \lor \neg\texttt{piece}(s(0), 0, P_4) \lor \neg\texttt{piece}(s(s(0)), 0, P_4)$
- R5: $\texttt{win}(P_5) \lor \neg\texttt{piece}(0, s(0), P_5) \lor \neg\texttt{piece}(s(0), s(0), P_5) \lor \neg\texttt{piece}(s(s(0)), s(0), P_5)$
- R6: $\texttt{win}(P_6) \lor \neg\texttt{piece}(0, s(s(0)), P_6) \lor \neg\texttt{piece}(s(0), s(s(0)), P_6) \lor \neg\texttt{piece}(s(s(0)), s(s(0)), P_6)$
- R7: $\texttt{win}(P_7) \lor \neg\texttt{piece}(0, 0, P_7) \lor \neg\texttt{piece}(s(0), s(0), P_7) \lor \neg\texttt{piece}(s(s(0)), s(s(0)), P_7)$
- R8: $\texttt{win}(P_8) \lor \neg\texttt{piece}(0, s(s(0)), P_8) \lor \neg\texttt{piece}(s(0), s(0), P_8) \lor \neg\texttt{piece}(s(s(0)), 0, P_8)$

Query (Q):

- `win(nought).`

Negated Query (NQ):

- ¬`win(nought).`

Resolution Steps:
Step 1: Resolve NQ

- **Q:** ¬`win(nought).`
- **R**(R4): `win(P_4)` ∨¬`piece(0, 0, P_4)` ∨¬`piece(s(0), 0, P_4)` ∨¬`piece(s(s(0)), 0, P_4).`
- **Θ:** {`P_4 | nought`}
- **QΘ:** ¬`win(nought).`
- **RΘ:** `win(nought)` ∨¬`piece(0, 0, nought)` ∨¬`piece(s(0), 0, nought)` ∨¬`piece(s(s(0)), 0, nought).`
- **Resolvent (C1):** ¬`piece(0, 0, nought)` ∨¬`piece(s(0), 0, nought)` ∨¬`piece(s(s(0)), 0, nought).`

Step 2: Resolve C1

- **Q:** ¬`piece(0, 0, nought)` ∨¬`piece(s(0), 0, nought)` ∨¬`piece(s(s(0)), 0, nought).`
- **R**(F1): `piece(0, 0, nought).`
- **Θ:** {}
- **Resolvent (C2):** ¬`piece(s(0), 0, nought)` ∨¬`piece(s(s(0)), 0, nought).`

Step 3: Resolve C2

- **Q:** ¬`piece(s(0), 0, nought)` ∨¬`piece(s(s(0)), 0, nought).`

- **R**(F4): `piece(s(0), 0, nought).`
- **Θ:** {}
- **Resolvent (C3):** ¬`piece(s(s(0)), 0, nought).`

Step 4: Resolve C3

- **Q:** ¬`piece(s(s(0)), 0, nought)`
- **R**(F6): `piece(s(s(0)), 0, nought).`
- **Θ:** {}
- **Resolvent:** `[].` (Empty Clause)

The derivation of the empty clause proves that `win(nought)` is true.

Task 5: LP - Fibonacci sequence (2.5 P.)

1. Define the predicate **fib(X)**, which is true for all natural numbers contained in the fibonacci sequence[2]. Use the *successor* definition (e.g. s(0), s(s(s(0))) to represent natural numbers and the *plus(X,Y,Z)* predicate discussed in the lecture.
   Remark: There are different definitions concerning the first two values of the fibonacci sequence. It is **mandatory** to use the definition with (1,1) as the first two values.
   Hint: You can define helper predicates as needed.

   ```
   (P1) plus(0,Y,Y).
   (P2) plus(s(X), Y, s(Z)) := plus(X,Y,Z).
   ```

2. Argue the trueness of the following query using resolution. Write down query, rule/fact and the substitution for every step.

   ```
   := fib(s(s(0))).
   ```

Solution

5.2.1 Define the predicate `fib(X)`

The successor and the plus(X,Y,Z) predicate is taken from the lecture, so i wont define them here again.

Before we define the predicate fib(X), first lets create a helper predicate "fib_next(X, Y)" which is true when Y is the next Fibonacci number in sequence after X, which is also a Fibonacci number:

- `fib_next(s(0), s(0)).`
  This is the base case: the first two consecutive Fibonacci numbers are (1, 1).
- `fib_next(X, Y) :- fib_next(Z, X), plus(Z, X, Y).`
  This is a recursive rule. It checks that the X is a Fibonacci number (by being the next fibonacci number of another fibonacci number Z). Then it checks that Y is a valid next Fibonacci number, if it is sum of X and Z.

Now we can define the `fib(X)` predicate.

- `fib(X) :- fib_next(_, X).`
  Number 'X' is a Fibonacci number if it appears in any of the fib_next. However this won't work if X is the first fibonacci number s(0), so we also need the base case for fib(X)
- `fib(s(0)).`
  This is the base case stating that the number 1 (`s(0)`) is a Fibonacci number

5.2.2 Argue the trueness of `:= fib(s(s(0)))` using resolution.

Facts (F):

- F1: `fib_next(s(0), s(0))`
- F2: `fib(s(0))`

Rules (converted to clauses, separate namespace for each rule):

- R1: `plus(0, Y_1, Y_1).` (from VO)
- R2: `plus(s(X_2), Y_2, s(Z_2))` ∨¬`plus(X_2, Y_2, Z_2).` (from VO)
- R3: `fib(X_3)` ∨¬`fib_next(A_3, X_3)`
- R4: `fib_next(X_4, Y_4)` ∨¬`fib_next(Z_4, X_4)` ∨¬`plus(Z_4, X_4, Y_4).`

Query (Q):

[2] https://en.wikipedia.org/wiki/Fibonacci_sequence

- `fib(s(s(0)))`.

Negated Query (NQ):

- ¬`fib(s(s(0)))`.

Resolution Steps:

Step 1: Resolve NQ

- **Q:** ¬`fib(s(s(0)))`.
- **R**(R3): `fib(X_3)` ∨¬`fib_next(A_3, X_3)`.
- **Θ:** {`X_3 | s(s(0))`}
- **QΘ:**¬`fib(s(s(0)))`.
- **RΘ:** `fib(s(s(0)))` ∨¬`fib_next(A_3, s(s(0)))`.
- **Resolvent (C1):** ¬`fib_next(A_3, s(s(0)))`.

Step 2: Resolve C1

- **Q:** ¬`fib_next(A_3, s(s(0)))`
- **R(R4):** `fib_next(X_4, Y_4)` ∨ ¬`fib_next(Z_4, X_4)` ∨ ¬`plus(Z_4, X_4, Y_4)`
- **Θ:** {`A_3 | X_4, Y_4 | s(s(0))`}
- **QΘ:**¬`fib_next(X_4, s(s(0)))`
- **RΘ:** `fib_next(X_4, s(s(0)))` ∨ ¬`fib_next(Z_4, X_4)` ∨ ¬`plus(Z_4, X_4, s(s(0)))`
- **Resolvent (C2):** ¬`fib_next(Z_4, X_4)` ∨ ¬`plus(Z_4, X_4, s(s(0)))`

Step 3: Resolve C2

- **Q:** ¬`fib_next(Z_4, X_4)` ∨ ¬`plus(Z_4, X_4, s(s(0)))`
- **R**(F1): `fib_next(s(0), s(0))`.
- **Θ:** {`Z_4 | s(0), X_4 | s(0)`}
- **QΘ:** ¬`fib_next(s(0), s(0))` ∨ ¬`plus(s(0), s(0), s(s(0)))`
- **RΘ:** `fib_next(s(0), s(0))`.
- **Resolvent (C3):** ¬`plus(s(0), s(0), s(s(0)))`

Step 4: Resolve C3

- **Q:** ¬`plus(s(0), s(0), s(s(0)))`
- **R(R2):** `plus(s(X_2), Y_2, s(Z_2))` ∨¬`plus(X_2, Y_2, Z_2)`.
- **Θ:** {`X_2 | 0, Y_2 | s(0), Z_2 | s(0)`}
- **QΘ:** ¬`plus(s(0), s(0), s(s(0)))`
- **RΘ:** `plus(s(0), s(0), s(s(0)))` ∨¬`plus(0, s(0), s(0))`.
- **Resolvent (C4):** ¬`plus(0, s(0), s(0))`.

Step 5: Resolve C4

- **Q:** ¬`plus(0, s(0), s(0))`
- **R**(R1): `plus(0, Y_1, Y_1).`
- **Θ:** {`Y_1 | s(0)`}
- **QΘ:** ¬`plus(0, s(0), s(0))`
- **RΘ:** `plus(0, s(0), s(0)).`
- **Resolvent:** `[].` (Empty Clause)

The successful derivation of the empty clause '[]' means that our original query `fib(s(s(0)))` is true

Task 6: LP incl. Lists (3 P.)

The predicate **bld(V, R)**, describes a list containing **Value V** and the **Tail R**. **Empty Lists** are represented by the **Constant nil**. Lists may contain any constant and are zero-indexed.

1. (2 P.) The predicate **indexof(V, L, I)** should be true, if the list L contains the constant V at index I.
   - Define all necessary rules for the predicate.
   - Use resolution **with backtracking** to calculate **all possible values** for C. Label every step.

```
:= indexof(a,bld(a,bld(b, bld(a, nil))), C)
```

Solution:

The rules for *indexof* are:
(I1): $indexof(V, bld(V, X), 0).$
(I2): $indexof(V, bld(X, R), s(I)) :- \ indexof(V, R, I).$

The query is:
$Q : \neg indexof(a, bld(a, bld(b, bld(a, nil))), C)$

1. $G : \neg indexof(a, bld(a, bld(b, bld(a, nil))), C)$
   $Step(A1), R(I1) : indexof(V_1, bld(V_1, X_1), 0).$
   $\Theta_1 = \{V_1|a, X_1|bld(b, bld(a, nil)), C|0\}$
   $Sub.G. : \neg indexof(a, bld(a, bld(b, bld(a, nil))), 0)$
   $Sub.R. : indexof(a, bld(a, bld(b, bld(a, nil))), 0)$
   $\square$
   We derived $\square$, thus a possible variable instantiation can be read from Θ_1: $C = 0$.
2. Backtrack to find more solutions.
   $G : \neg indexof(a, bld(a, bld(b, bld(a, nil))), C)$
   $Step(A2), R(I2) : indexof(V_2, bld(X_2, R_2), s(I_2)) \vee \neg indexof(V_2, R_2, I_2).$
   $\Theta_2 = \{V_2|a, X_2|a, R_2|bld(b, bld(a, nil)), C|s(I_2)\}$
   $Sub.G. : \neg indexof(a, bld(b, bld(a, nil)), I_2)$
   $Sub.R. : indexof(a, bld(a, bld(b, bld(a, nil))), s(I_2)) \vee \neg indexof(a, bld(b, bld(a, nil)), I_2)$
3. Now, we need to resolve $\neg indexof(a, bld(b, bld(a, nil)), I_2)$.
   $G : \neg indexof(a, bld(b, bld(a, nil)), I_2)$
   $Step(B1), R(I2) : indexof(V_3, bld(X_3, R_3), s(I_3)) \vee \neg indexof(V_3, R_3, I_3).$
   $\Theta_3 = \{V_3|a, X_3|b, R_3|bld(a, nil), I_2|s(I_3)\}$
   $Sub.G. : \neg indexof(a, bld(a, nil), I_3)$
   $Sub.R. : indexof(a, bld(b, bld(a, nil)), s(I_3)) \vee \neg indexof(a, bld(a, nil), I_3)$
4. Now, we need to resolve $\neg indexof(a, bld(a, nil), I_3)$.
   $G : \neg indexof(a, bld(a, nil), I_3)$
   $Step(C1), R(I1) : indexof(V_4, bld(V_4, X_4), 0).$
   $\Theta_4 = \{V_4|a, X_4|nil, I_3|0\}$
   $Sub.G. : \neg indexof(a, bld(a, nil), 0)$
   $Sub.R. : indexof(a, bld(a, nil), 0)$
   $\square$

   Note: Applying Rule (I2) to $\neg indexof(V, R, I)$ with the substituted values, which yields $\neg indexof(a, nil, I)$, would not work because the list nil is not of the form bld(X,R).

   $\rightarrow$ We derived $\square$. A variable instantiation can be read from the composition of Θ_2, Θ_3, and Θ_4:
   $C\Theta_2\Theta_3\Theta_4 = s(I_2)\Theta_3\Theta_4 = s(s(I_3))\Theta_4 = s(s(0)).$

Thus, the possible variable instantiations for C are:
$C = 0$
$C = s(s(0))$

Task 7: Modeling in LP (Bonus) (1 − 3 P.)

The Eurocity 164 ("Transalpin") is a direct train connection between Graz and Zurich[3]. The train leaves Graz daily at 9:45AM and arrives in Zurich at 7:30PM, stopping at the following stops: **Graz Hbf. - Leoben Hbf. - Selzthal - Bischofshofen - Schwarzach St.Veit - Innsbruck - Feldkirch - Buchs SG - Zürich HB**.

In May 2025 there is the following (scheduled) coach sequence[4]:

Number	Name
312	Bmz^{73}
311	$WRmz^{73}$
310	$ADbmpsz^{73}$
309	Apm
308	Bpm_{2090b}
307	Bpm_{2090b}
306	Bpm_{2090}
305	Bpm_{2090}
304	Bpm_{2090b}
303	$Bmpz^{70}$

Coach Name	Class	Type	Reservable Seats		
			Window	Middle	Hallway
Bmz^{73}	second	compartment	15	13	11
$WRmz^{73}$	-	dining car	-	-	-
$ADbmpsz^{73}$	first	open-plan	15	-	13
Apm	first	panorama car	11,15	-	13
Bpm_{2090b}	second	open-plan	15	17	11
Bpm_{2090}	second	open-plan	11,15	-	17
$Bmpz^{70}$	second	open-plan	11,15	-	17

Table 2: Coach sequence and coach types

On Friday, June 6, 2025, a large part of the train from Graz to Zürich is reserved. Only the following seats are available in the specified sections:

Coach	Seat	Free (from-to)
303	11	Selzthal - Schwarzach St. Veit
303	17	Graz - Zürich
304	17	Innsbruck - Buchs SG
309	13	Graz - Feldkirch
310	15	Graz - Zürich
312	11	Leoben - Bischofshofen

Table 3: Free Seats

Write a reservation system for EC 164 in **LP** that offers the following options:

- Selection of class (first, second)
- Selection of coach type (compartment, open-plan, panorama car)
- Selection of seat type (window, middle, hallway)
- Entry (station name)
- Exit (station name)

Your system should return one available coach and seat number based on these parameters.

[3] `https://en.wikipedia.org/wiki/Transalpin`

[4] A colored description of the given tables can be found here: `https://www.vagonweb.cz`. Note that coach 302 should not be modeled.

Remarks:

- Only the aforementioned coaches, stations and seat numbers must be supported.
- You can take the predicate *different(X, Y)*, which is true for each pair at different stations, as given.
- Your submission must be written in LP! You can prepare or check your concept by writing it in Prolog, but keep in mind that LP does not support several operators of Prolog (including `\=`).
- It is not required to do a resolution, but it should be possible in theory. Rules like `connected(X,Y) := connected(Y,X)` create infinite recursions during resolution and are therefore not allowed.
- Your model will be graded for correctness and extensibility (think about the effort to model another train with a different set of coaches). Try to describe the system with as few but precise rules and facts as possible. The reference solution contains < 60 facts and rules.